

An Adaptive Risk Control and Security Management for Embedded Real-Time System

Yue Ma¹, Wei Jiang^{1*}, Nan Sang¹, Ziguo Zhong²

¹School of Computer Science and Engineering
University of Electronic Science and Technology of China
yue_ma_880131@hotmail.com, {weijiang, sn}@uestc.edu.cn

*Corresponding author

²Computer Science and Engineering
University of Nebraska – Lincoln, NE, 68508, USA
zzhong@cse.unl.edu

Abstract—Traditional mechanisms for risk control and task scheduling may not be very suitable for security-critical embedded real-time systems under dynamic operation environments. In this paper, we are interested in design of an adaptive risk control and management mechanism of the uniprocessor system for real-time embedded applications. With the purpose of obtaining adaptive capability to reduce system risks without sacrificing soft real-time performance, a two-level feedback scheduling framework was deployed at runtime. The upper-level component automatically monitors the status of the waiting queue, then admits or rejects the input workloads and assigns a reasonable security level for ready tasks. The lower-level scheduling framework can dynamically adjust the security level of ready tasks based on the runtime situation. Simulation results show the superiority of the proposed mechanism, especially the good adaptivity under dynamic system workloads.

Keywords—security risk management; feedback control; embedded real-time system; scheduling

I. INTRODUCTION

Embedded real-time systems are facing more challenges in risk management and security guarantee. Firstly, embedded real-time systems are increasingly deployed in the open environments where operating conditions are dynamic and security threatened. Secondly, embedded systems are battery-powered and even under no nursing state [1]. Quick energy consumption or early exhaustion of battery may lead to failure of critical task, resulting in degeneration of security quality. Thus, for security-critical real-time applications, there is a strong need to meet multiple constraints including real-time, secure and energy efficiency simultaneously. This situation is more serious when the input workloads cannot be estimated. Therefore, it is imperative and significant to design a good control and management mechanism for security-critical real-time applications under dynamic and resource-limited environments.

Scheduling is a good approach to obtain high performance for embedded real-time systems. Formerly, scheduling algorithms proposed for real-time systems are aimed for static and priority based scheduling mechanism and have been proven to be efficiently and widely used in many fields [2]. Unfortunately, these works are based on the assumption of knowing most useful information of applications like worst case execution time. These mechanisms are designed in “open-loop” mode and cannot be adjusted based on un-predicted situations, which result in poor performance under dynamic environment unpredicted. This situation could be more severe when

real-time systems, such as security-critical embedded web servers, run in the network environment where the potential threats cannot be modeling accurately. To guarantee the system performance in such unpredicted environments, some feedback-based adaptive QoS scheduling algorithms have been developed as a cost-effective approach for real-time systems [3]. Nowadays, this design method has been applied in many fields, such as thermal control [4], fault-tolerance [5] and dynamic voltage scaling [6].

Recently, many researchers have focused on risk and security management systems and proposed many useful algorithms and models. Gehani et al. introduce a formal model to characterize the risk faced by a host connected to the Internet and also describe how the risk can be managed in real-time by adapting the exposure [7]. S. Markose et al. develop a structured and object-oriented methodology for security requirements analysis [8]. Gebotys et al. combined the security performance and energy-efficiency then proposed a low-power security optimization algorithm that unfortunately is a static policy and not adaptive to the unpredicted surroundings [9]. However, most of these researches have some weakness, such as lacking of energy analysis or adaptivity.

In this paper, we are interested in design of adaptive risk control and management mechanism of the uniprocessor system for real-time embedded applications. Considering the aperiodic real-time applications, we are trying to propose an adaptive risk-aware real-time scheduling mechanism for security-critical uniprocessor systems. System administrator just needs to appoint the expected system performance, and this mechanism can automatically monitor system status and adjust its performance to satisfy these demands. To meet this requirement, there should be an advanced mechanism to manage tasks, i.e. admitting or rejecting aperiodic tasks. In addition, this framework should also have the ability to dynamically adjust security level of tasks in the waiting queue. Inspired by the automatic control related work in storage system [10] and real-time system [11], a two-level feedback control scheme is employed to resolve the problem in this paper. The upper-level mainly admits or rejects the newly arrival task and initialize an appropriate security level for it. The lower-level dynamically adjust this level for ready tasks to maintain the real-time and security at the runtime.

The primary contributions of this paper are:

1. introducing security-critical applications based on data-acquisition embedded server with uniprocessor system;

2. establishing the security-aware task model, security overhead model and security risk metric for aperiodic real-time applications;
3. combining the soft real-time and security requirements in one framework and deploying a proportional controller based MIMO scheduling mechanism.

The rest of this paper is organized as follows. The motivating application and feedback architecture is discussed in Section II. Section III presents the system model and Section IV demonstrates the risk control scheduling algorithm. In Section V, we conduct simulations and analyze the experimental results. Finally, conclusions are drawn in Section VI.

II. MOTIVATING APPLICATIONS AND SYSTEM ARCHITECTURE

A. Motivating Scenario

This part describes a typical data-acquisition system that we use as a case study to investigate adaptive risk control and management mechanism in an embedded system [12]. The typical architecture of data-acquisition system is shown in Fig. 1. In response the request from clients, embedded server acquires the information from specific sensors and then feeding them back. In most cases, the communication link is Internet or other public networks, which means this communication between clients and sever is not totally security. Hence, obtained data should be encrypted to avoid intercepting even when hackers enter into the network.

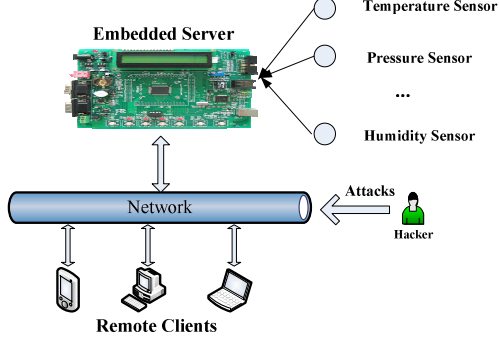


Fig. 1. Typical Data-Acquisition Architecture

Based on this motivational application, we make some assumptions as follows.

1. For the direction of most data in data-acquisition system is from server to client, we only consider data encryption of embedded server.
2. Considering the internet protocol, such as SSL, has supported many cryptography algorithms, we assume all the data can be encrypted by any of these algorithms.
3. In our research, we do not consider the energy and time cost during the switch between different cryptographic algorithms.

B. Feedback Scheduling Architecture

To meet the requirements discussed in Section I, two goals have been set for our proposed mechanism. Goal 1: adapting to fluctuations in input workload. For the arrival

time of new aperiodic task is unpredictable and not all input workload can be responded, it is necessary to decide how to reject or admit new tasks. Moreover, to decrease the system overhead at runtime, it's better to assign an appropriate initial security level for new tasks before processed. Goal 2: adaptive to runtime status and changing of resource availability. For the character of embedded systems, the runtime status and available system resources are variable and cannot be accurately estimated. To make the system working as we expected, this mechanism should dynamically monitor system status and modify security level for each task.

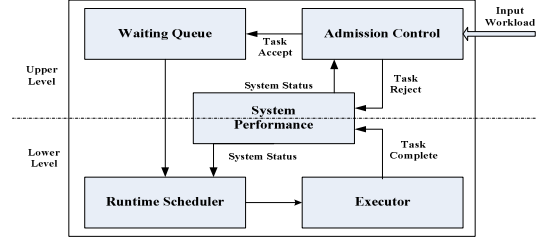


Fig. 2. The Two-Level Scheduling Framework

We design a two-level feedback scheduling framework to meet these goals (in Fig. 2). In the upper level, *Admission Control* admits tasks and initializes their security levels. The tasks, which might miss their deadline at a high possibility, will be rejected to serve. The decisions of *Admission Control* rest on system status monitored by *System Performance*, and in turn, these decisions can affect the system status too. In the lower level, *Runtime Scheduler* accurately adjusts the security level for tasks based on system status at each sampling window. The idler the system is, the stronger security protection will be provided. Finally, *Executor* is an execution component.

III. SYSTEM MODEL

A. Security Overhead Model

Confidentiality is the most popular requirement for information security, and it means the secrecy of data being expected by communicating parties is maintained. To research the confidentiality of embedded system, one prerequisite is to investigate the performance of cryptography algorithms, including time and energy cost. Many researchers have measured and analyzed the time and energy cost of some symmetric algorithms [13] [14], but we doubt the results may not suit for embedded system for these studies based on specific protocols rather than real embedded devices. In this paper, we measure the time and energy consumptions of encryption for some symmetric algorithms in a real embedded platform. This measurement is based on a real-time operation system, $\mu\text{C}/\text{OS-II}$ and an open source cryptographic library named Cryptlib [15]. The measure instrument is PXI 1024Q with a NI 6221 data acquisition card [16], and the target device is an embedded development board shipped with S3C2440 ARM [17] processor and owning 64MB SDRAM.

The time and energy costs of symmetric algorithms are listed in Table I and II. Note that these measured data are relied on our specific environment, and maybe differ from other theoretical researches [18]. It is obvious that the data

size of plaintext is one decisive factor for time cost of encryption. Some researchers have established a liner relation between time cost and number of encrypting round [19], and we think it is still reasonable to establish a liner function to describe the relationship between time cost and data size.

$$Tcry_i = Wcry_i + Rcry_i \times S \quad (1)$$

The time cost of i^{th} cryptography algorithm $Tcry_i$ is proportional to the size of plaintext, S , adding a coefficient, $Wcry_i$. For example, $Wcry_{RC4} = 2.24$ and $Rcry_{RC4} = 0.004$.

Undoubtedly, the energy efficiency can also affect the algorithm's performance, but the concrete effect is still an open problem. Like the energy/quality concept proposed by J. Peddersen [20], the energy/time trade-off is used in this paper to present the energy cost per millisecond of execution time. In the processing to compute this value for every algorithm, we find an interesting phenomenon that on average, the energy/time trade-offs of all the algorithms are briefly equal to 0.32 mJ/ms. In another word, not matter for RC4, RC5 or any other one, algorithms executing one millisecond will consume the energy nearly 0.32mJ. Though these obtained data might be depended on our hardware platform, we recognize this proportional relationship between energy and time cost must be widely existed in different platforms. Hence, for this fixed proportional relationship, we do not need to design a specialized method to maintain the energy-efficiency. Essentially, the approach to guarantee the real-time can limit the energy consumption. We recognize this relationship between real-time and energy cost is very useful for future research.

We also measure the time and energy costs of key initialization, which are shown in Table III. Note the key sizes of all cryptographic algorithms are default in Cryptlib. Comparing with encryption execution, the processing to initialize a key consumes more energy and time. Hence, we assume all the required key has been established and

distributed to every authenticated clients when the system initializing.

TABLE III. Time and Energy cost of Key Initialization

Algorithm	Time (mJ)	Energy (mJ)
RC4	124.2	39.99
RC5	123.7	39.96
BLOWFISH	157.8	50.85
CAST	123.6	39.86
AES	123.6	39.83
IDEA	124.9	40.23
SKIPJACK	127.2	40.95

Based on these measured data, we now introduce the concept of Security Level (SL) to describe and distinguish these cryptography algorithms. In Table IV, AT_i represents the average time and energy cost when data size altering between 16B and 8KB, respectively. Similar with [21], the security level of an algorithm depends on the time cost or execution speed. We assign security level 1 to the weakest yet fastest encryption algorithm, RC4. Security levels for the rest of the algorithms can be computed by (2),

$$SL_i = AT_i / 1.87, i \leq 1 \leq 7 \quad (2)$$

TABLE IV. Security Levels of Cryptographic Algorithms

Algorithm	AE_i (mJ)	SL_i
RC4	1.87	1.00
RC5	3.76	2.01
BLOWFISH	5.03	2.69
CAST	5.28	2.87
AES	5.73	3.06
IDEA	5.75	3.07
SKIPJACK	6.34	3.40

TABLE I. Time Costs of Encryption of Symmetric Algorithms

Time (ms)	16B	32B	64B	128B	256B	512B	1024B	2048B	4096B
RC4	2.30	2.36	2.50	2.77	3.27	4.22	6.28	10.25	18.30
RC5	4.90	5.03	5.29	5.77	6.72	8.65	12.50	20.23	35.70
BLOWFISH	4.98	5.15	5.55	6.31	7.83	10.88	16.98	29.17	53.57
CAST	5.00	5.17	5.61	6.40	8.07	11.28	17.83	30.85	56.90
AES	5.04	5.24	5.73	6.64	8.44	12.12	19.43	34.07	63.32
IDEA	5.01	5.24	5.72	6.66	8.51	12.22	19.61	34.49	64.20
SKIPJACK	5.07	5.29	5.86	6.92	9.03	13.24	21.72	38.54	72.35

TABLE II. Energy Costs of Encryption of Symmetric Algorithms

Energy (mJ)	16B	32B	64B	128B	256B	512B	1024B	2048B	4096B
RC4	0.74	0.75	0.80	0.89	1.05	1.35	2.02	3.31	5.92
RC5	1.57	1.61	1.70	1.85	2.16	2.79	4.04	6.54	11.56
BLOWFISH	1.60	1.65	1.78	2.02	2.51	3.50	5.47	9.41	17.30
CAST	1.60	1.66	1.80	2.05	2.60	3.64	5.76	9.98	18.43
AES	1.61	1.68	1.84	2.13	2.71	3.90	6.26	10.99	20.43
IDEA	1.60	1.67	1.83	2.13	2.72	3.91	6.28	11.05	20.57
SKIPJACK	5.07	5.29	5.86	6.92	9.03	13.24	21.72	38.54	72.35

The major advantage of this approach is that we can easily compute the approximate time cost of a cryptography algorithm by its security level. For example, if the data size is S , the time cost of AES can be computed as,

$$Tcry_{AES} = SL_{AES} \times (Wcry_{RC4} + Rcry_{RC4} \times S) \quad (3)$$

or,

$$Tcry_{AES} = SL_{AES} \times (2.24 + 0.004 \times S)$$

In addition, for the one-to-one relationship between security level and cryptography algorithm, we can use the security level of an algorithm to refer to the algorithm itself.

B. Modeling Security Risk

According to the job failure model of [22], we set P_i as the Risk Ratio (or the failure probability) of the i^{th} task. The risk ratio can be expressed as,

$$P_i = 1 - e^{\alpha(SL_i - 3.40)}, \quad 1 \leq SL_i \leq 3.40 \quad (4)$$

where α is the risk related co-efficient. We make an assumption that $P_i = 0$ when the security protection is the strongest with the maximal level 3.40.

We use Risk Loss (**RL**) to indicate the system loss when the security protection being invalid under the inside or outside threats. It can be calculated as (5).

$$RL = \sum_{i=1}^n P_i \cdot w_i \quad (5)$$

w_i is the intrinsic character of the i^{th} tasks and represents the lost value when security protection fails.

Note that this model assumes that risk ratio and risk loss are a function of security level, and also relative to cryptographic algorithms' energy and time consumption. In addition, the risk ratio we discussed here is a theoretical probability to describe the failure possibility of security protecting mechanism.

C. Task Model

This paper focuses on the aperiodic applications for uniprocessor system. Each task arrives arbitrarily and has strict real-time constraint, which is described by $T_i = \{a_i, s_i, e_i, d_i, S_i, SL_i\}$. a_i represents the arrival time of task T_i , and s_i is the start time of execution. e_i and d_i are the execution time and deadline, respectively. All the time related parameters are illustrated by Fig. 3.

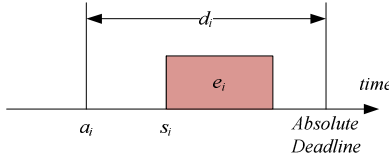


Fig. 3. The Illustration of an Aperiodic Task

S_i and SL_i represent the data size of plaintext and the security level of the i^{th} task. The maximum level $SL_{\max} = 3.4$, and minimum $SL_{\min} = 1.0$. In our research, we mainly concentrate the effect of security protection on real-time feature. It is can be assumed that the execution time e_i is nearly close to the cryptographic algorithm execution time. For our method to assign security level, e_i can be computed as,

$$e_i = SL_i \times (Wcry_{RC4} + Rcry_{RC4} \times S_i) \quad (6)$$

For each aperiodic task, it can be delayed to execute for some time (slack time) after arrival. We utilize Slack Ratio (**SR**) to describe the task's relaxation degree. Certainly, bigger slack ratio brings more flexibility.

$$SR_i = 1 - (s_i - a_i + e_i) / d_i \quad (7)$$

IV. RISK CONTROL AND MANAGEMENT ALGORITHMS

In this section, we mainly discuss the algorithms deployed in our mechanism. In part A, we talk about how to admit a newly come task. Part B and part C discuss the upper limit of initialized security level. The algorithm in lower-level feedback is introduced in part D.

A. Tasks Admission

To guarantee the soft real-time for most tasks, system should refuse to serve for the tasks which cannot meet their deadlines at a high possibility. First of all, we assume the new task will be inserted in the i^{th} place of waiting queue if it can be admitted. We choose priority-based queuing strategy, which means shorter deadline tasks have higher priorities and stay at the front of queue. Undoubtedly, no matter the new task be rejected or admitted, security levels of other tasks in the waiting queue cannot be changed.

To accept as more new tasks as possible, the only requirement to admit a new task T_i is whether the task can meet its deadline,

$$s_i + e_i \leq d_i$$

From the queuing strategy, it can be transformed as,

$$r + \sum_{j=1}^{i-1} e_j + e_i \leq d_i \quad (8)$$

r represent the remaining execution time of T_{cur} , the current processed task, and e_j is the execution time of j^{th} task in the front of queue. If the new task T_i cannot meet this requirement, system could refuse to serve for it. Though this policy neither guarantees the real-time of new tasks nor waiting tasks, it has the ability to reject tasks which are most likely to miss their deadlines.

B. Deadline Guarantee

To reduce the system overhead at runtime, the *Admission Control* should assign a reasonable initial security level for new tasks. The initialized security level of task T_i should be as high as possible but not make the deadline missed.

First of all, we transform (8) as,

$$e_i \leq d_i - r - \sum_{j=1}^{i-1} e_j \quad (9)$$

From (6), task's execution time is a function of security level and data size, so the above inequality can be expanded as

$$\begin{aligned} & SL_i \times (Wcry_{RC4} + Rcry_{RC4} \times S_i) \\ & \leq d_i - r - \sum_{j=1}^{i-1} SL_j \times (Wcry_{RC4} + Rcry_{RC4} \times S_j) \end{aligned} \quad (10)$$

Hence, initialization of security level can be seemed as a process to find the highest level under the constraint in (10).

Unfortunately, the final security level of task T_j is decided by the lower-level runtime scheduling, which means SL_j might be changed before being executed. So we need to do some prediction. From feedback control basic theory, system status in the last window is instructive in the current window. Roughly, it can be estimated as

$$\forall 1 \leq j \leq (i-1), SL_j \approx \overline{SL}$$

where $\overline{SL}$ represents the average security level in the last sampling window. So the equation (10) can be simplified as follows,

$$SL_i \times (Wcry_{RC4} + Rcry_{RC4} \times S_i) \leq d_i - r - (n-1) \times \overline{SL} \times (Wcry_{RC4} + Rcry_{RC4} \times S_j) \quad (11)$$

It can be transformed as,

$$SL_i \leq \frac{d_i - r - (n-1) \times \overline{SL} \times (Wcry_{RC4} + Rcry_{RC4} \times S_j)}{Wcry_{RC4} + Rcry_{RC4} \times S_i} \quad (12)$$

Inequality (12) gives an upper limit of security level to satisfy the real-time. Note that it is a predicted expression and still cannot ensure the real-time strictly. But the task can satisfy its real-time requirement with a high possibility. Actually, for the system which operating environments cannot be accurately characterized, there is no method to guarantee all the aperiodic tasks can meet their real-time requirements.

C. Online Slack Adjustment

The tasks in the front of waiting queue are not affected by the new task. But, when a new task arriving and being inserted into the waiting queue, the start time of tasks with a lower priority in the waiting queue must be changed. The algorithm is here used to evaluate such an influence and tries to limit its impact within a reasonable range.

Based on our definition of slack ratio, when a new task admitted, the slacks of tasks in the tail of queue will be changed as equation (13).

$$SR_k = 1 - (s_k - a_k + e_k + e_i) / d_k, \text{ for } \forall k > i \quad (13)$$

The relationship between SR_k and e_i is negative, and when e_i increased, SR_k will be decreased, and even lower than 0 (deadline missed). Therefore, another upper limit of initialized security level is to guarantee the SR_k lower than a desired threshold SR^S .

$$(s_k - a_k + e_k + e_i) / d_k \geq SR^S, \text{ for } \forall k > i$$

From the definition of e_i , it can be transformed as,

$$SL_i \leq \frac{SR^S \times d_k - s_k + a_k - e_k}{Wcry_{RC4} + Rcry_{RC4} \times S_i} \quad (14)$$

Both of (12) and (14) are the upper limit of initialized security level and deployed in the component *Admission Control*. The flow chart in Fig. 4 virtualizes this function.

In left part of this flow chart, Admission Control makes a decision to admit or reject a new task. The middle part of this flow chart tries to increase the security level. If the security level cannot satisfy one of these three requirements, this process will be ended. But, no matter how this round ends, the security level must be higher than we expected, so we need to decrease it, as in the right part of this figure.

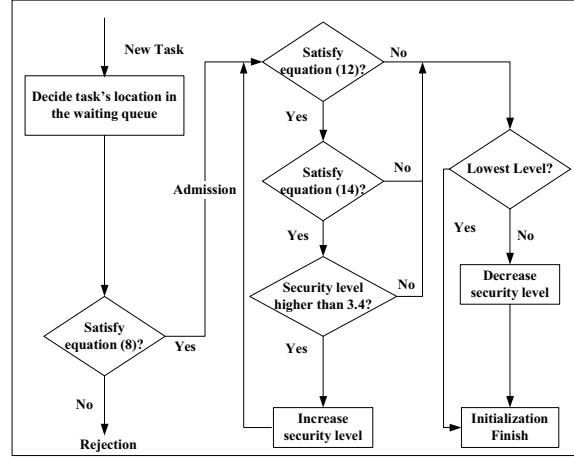


Fig. 4. Function of Admission Control

D. Feedback Control in the Lower-Level

The main purpose of the lower-level feedback is dynamically adjusting the security level based on the system status at run-time. Primarily based on the FCS proposed in [3], the system metrics in the lower-level are deadline miss ratio $M(k)$, risk ratio $R(k)$ and rejection ratio $REJ(k)$. $M(k)$ is defined as the number of deadline missed tasks divided by the total number of completed tasks and $REJ(k)$ equals the number of rejected tasks divided by the total number of aperiodic tasks. $R(k)$ is used to describe the risk faced by the whole system.

At every sampling window, the component System Performance collect the values of these metrics, and feed them back to three independent controller, *Miss Controller*, *Risk Controller* and *Rejection Controller*, respectively. These controllers compute a control input $e(k)$ to adjust the security level. The architecture is shown in Fig. 5.

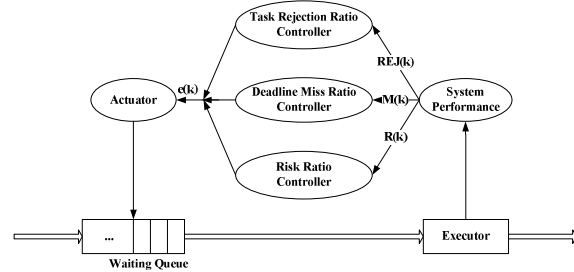


Fig. 5. The Framework of Runtime Scheduler

For these three controllers, we considered to use Proportional Controller instead of the traditional PID Controller which is complex and not very suitable for real-time system. The $e(k)$ can be computed as,

$$e(k) = K_M \cdot (M(k) - M_S) + K_R \cdot (R(k) - R_S) + K_{REJ} \cdot (REJ(k) - REJ_S) \quad (16)$$

where the K_M , K_R , and K_{REJ} are tunable parameter and M_S , R_S , REJ_S are their set points. *Actuator* (in Fig. 2) can adjust the security level based on the $e(k)$ for all the tasks in the waiting queue via some specific strategy appointed by system administrator.

We take the miss ratio control for example to discuss the closed-loop system model. The controlled variable in lower-level is the security level. The actuator changes the level $SL(k+1)$ according to the $e(k)$ at sampling window k ,

$$SL(k+1) = SL(k) + G_1 \cdot e(k) \quad (17)$$

where G_1 is called security-level ratio. Note in different window, the G_1 is changeable. However, for the controlling theory, it can be set as a constant value, and will not change the system performance.

Though there is no reason that $M(k)$ increase linearly with $SL(k)$, $SL(k)$ is proportional with CPU utilization, $u(k)$. Hence the relationship between $M(k)$ and $u(k)$ can be linearized at the vicinity of M_S as a factor G_2 , as in [3],

$$G_2 = \frac{dM(k)}{dSL(k)} \quad (18)$$

The above equation can be transformed as:

$$\begin{aligned} M(k) &= M(k-1) + G_2 \cdot (SL(k) - SL(k-1)) \\ &= M(k-1) + G \cdot e(k) \end{aligned} \quad (19)$$

where $G = G_1 \cdot G_2$. We convert this model to z -domain function for convenience of analysis,

$$M(z) = e(z) \cdot G / (z-1) \quad (20)$$

The control loop is shown in Fig. 6.

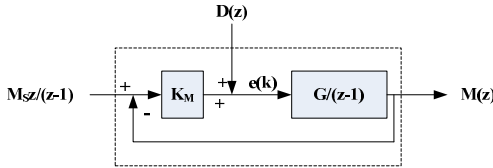


Fig. 6. Feedback Control Loop of Miss Ratio

$D(z)$ represents the disturb signal for miss ratio control. As mentioned in Fig. 3, the $e(k)$ is the output of three controllers, hence, $D(z)$ comes from the integrated output of risk ratio control and rejection ratio control. The transfer function of miss ratio control is,

$$T = \frac{G}{z - (1 - G \cdot K_M)} \quad (21)$$

From the control theory, the stability condition is the *poles* of transfer function in the unit circle of z -plane,

$$|1 - G \cdot K_M| < 1 \Leftrightarrow 0 < K_M < 2 / G \quad (22)$$

The other control loops of risk ratio and rejection ratio can be analyzed in the same way. Note for the candidate security levels are discrete and limited, the stability cannot be guaranteed strictly.

V. EXPERIMENTAL RESULTS AND DISCUSSION

A. Experiment Setup

As described above, a real-time system is subjected to 1) fluctuations in input workloads and 2) significant changes in system runtime status and environments. To validate our claim that this proposed mechanism can be risk-adaptive in real-time system, we evaluated the performance when: 1) input workloads were varied at runtime and 2) a significant drop in processing capacity. In this set of experiments, we assume the distribution of tasks' arrival time is approximated using a Poisson probability

distribution and the parameter is P_α . To make our experiments more accurately, we introduce the measured data of cryptography algorithms and suppose the execution time of one task, e_i , is enumerated in Table I with SL_i and S_i , which is a random value. Finally, the w_i (in equation 5) is also a random value in range of 0 to 10.

B. Experiment 1: Varying Input Workload

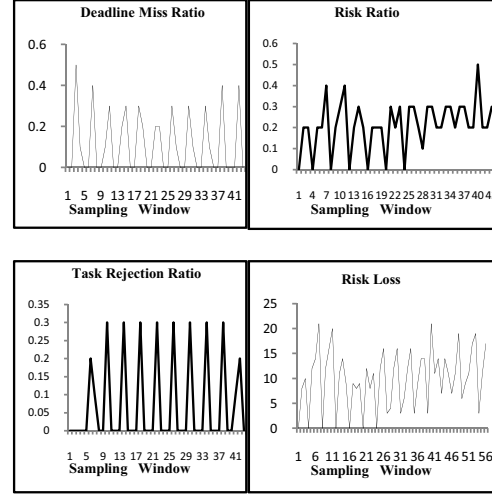


Fig. 7. System Performance when $P_\alpha = 0.8$

This experiment simulates the real-time computing node and demonstrates the adaptive risk control and management capabilities of our proposed mechanism. Firstly, we test the system performance of our mechanism when the Poisson parameter $P_\alpha = 0.8$. Based on the equation (5), we also calculate the sum of Risk Loss at every sampling window. The results are shown in Fig. 7. In this figure, it can be find that the system performance can fluctuate around our given criterion.

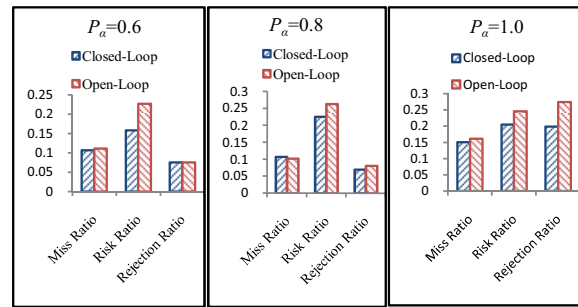


Fig. 8. System Comparison with Different P_α

To testify the advantages of our mechanism, we compare it with an open-loop system, as shown in Fig. 8. The major characteristic of open-loop system is that this system does not have the *Runtime Scheduler* component. The comparison metrics are the average value of Deadline Miss Ratio, Risk Ratio and Task Rejection Ratio. It can be clearly find that our mechanism has a better performance in every metric within the different P_α .

C. Experiment 2: Dropping in processing capacity

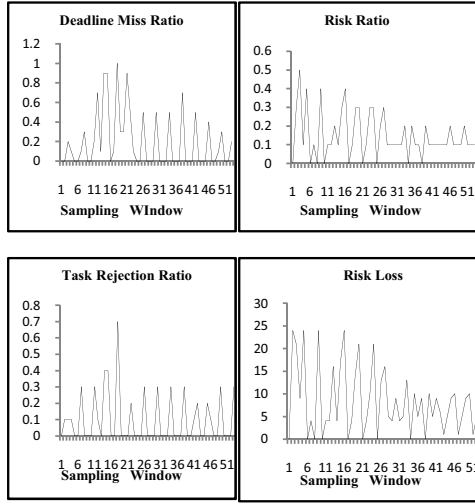


Fig. 9. System Performance When Processing Capacity Dropping

We do this experiment to evaluate system performance when the processing capacity suddenly dropping. We set $P_a = 0.8$ and assume that the processing capacity drops at the 12th sampling window and reverts at the 21th sampling window. During that time, the execution time of every processing task is doubled. From Fig. 9, it can be found that the $M(k)$ and $Rej(k)$ dramatically increased in the 12th sampling window. For our proposed mechanism trying to decrease the security level for each task, $M(k)$ and $Rej(k)$ are gradually decreased in this overloaded period. In addition, for more tasks being rejected in this period, the variation of $R(k)$ is relatively small. Thus, even when processing capacity dropping dramatically, system still faces low risk.

VI. CONCLUSIONS

In this paper, we focus on the security-critical real-time applications running on uniprocessor server system. We have devised an adaptive risk control and management mechanism which aims to reduce security risk as well as considering deadline miss ratio and task rejection ratio. To address the problem, a two-level feedback scheduling framework is proposed. A security risk metric is also proposed to assess the security quality of security-critical tasks. Considering the inherent characteristics of the embedded system, we measured the energy efficiency of different cryptographic algorithms and designed a new indicator to quantify the security performance. Experimental results demonstrate that the proposed mechanism can be adaptive to fluctuations in input workloads and runtime status. Finally, comparing with another open-loop control system, we testify that our mechanism has better overall performance by synthetic experiments.

VII. ACKNOWLEDGEMENT

This work was partly supported by the National Natural Science Foundation of China under Grant No. 61003032, the Fundamental Research Fund for the Central

Universities of China under Grant No. ZYGX2011J061 and the Research Fund of National Key Laboratory of Computer Architecture under Grant No. CARCH201104.

VIII. REFERENCES

- [1]. S. Ravi, A. Raghunathan, P. Kocher, and S. Hattangady, "Security in embedded systems: Design challenges," *ACM Transactions on Embedded Computing Systems*, vol. 3, pp. 461-491, 2004.
- [2]. L. Liu, J. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, pp. 46-61, 1973.
- [3]. C. Lu, J. A. Stankovic, G. Tao, S. H. Son, "Feedback Control Real-Time Scheduling: Framework, Modeling, and Algorithms," *Journal of Real-Time Systems*, Special Issue on Control-Theoretical Approaches to Real-Time Computing, 2002.
- [4]. Y. Fu, N. Kottenstette, "Feedback Thermal Control for Real-time Systems," 16th Real-Time and Embedded Technology and Applications Symposium, pp. 111-120, 2010.
- [5]. R. Sridharan, N. Gupta, R. Mahapatra, "Feedback-Controlled Reliability-Aware Power Management for Real-Time Embedded Systems," *ACM/IEEE on Design Automation Conference*, pp. 185-190, 2008.
- [6]. Y. Zhu, F. Mueller, "Feedback EDF Scheduling Exploiting Dynamic Voltage Scaling," *Real-Time and Embedded Technology and Applications Symposium*, pp. 84-93, 2004.
- [7]. A. Gehani, G. Kedem, "RheoStat: Real-Time Risk Management", In *Recent Advances in Intrusion Detection: 7th International Symposium*, pp. 296-314, 2004.
- [8]. S. Markose, X. Liu, B. McMillin, "A systematic framework for structured object-oriented security requirements analysis in embedded systems," *IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*, pp. 75-81, 2008.
- [9]. C. H. Gebotys, "Low Energy Security Optimization in Embedded Cryptographic Systems," *International Conference on Hardware and Software Codesign and System Synthesis*, pp. 224-229, 2004.
- [10]. J. Zhang, A. Sivasubramaniam, "Storage Performance Virtualization via Throughput and Latency Control," 13th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, pp. 135-142, 2005.
- [11]. T. He, J. Stankovic, "Feedback Control-based Dynamic Resource Management in Distributed Real-Time Systems," *Journal of Systems and Software*, vol. 80, pp. 997-104, 2007.
- [12]. Q. Zhou, W. Wu, and Y. Ma, "The embedded data acquisition system for Mössbauer spectrum," *Third Real-Time Linux Workshop Embedded Linux Expo & Conference, Real-Time Embedded & Computer Conference*, pp. 26-29, 2001.
- [13]. N. Potlapally, S. Ravi, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols," *IEEE Transactions on Mobile Computing*, vol. 5, pp. 128-143, 2005.
- [14]. O. Elkeelany, M. M. Matalgah, K. P. Sheikh, M. Thaker, "Performance Analysis of IPSec Protocol: Encryption and Authentication," *IEEE International Conference on Communications*, vol. 2, pp. 1164-1168, 2002.
- [15]. Peter Gutmann, *Cryptlib Security Toolkit Version 3.4*, <http://www.cryptlib.com>.
- [16]. National Instruments Corp, <http://www.ni.com>.
- [17]. Samsung Electronics Co., Ltd. S3C2440A 32-Bit CMOS Microcontroller User's Manual Revision 1.
- [18]. J. Wilbrink, D. Nativel, T. Morin, "Networked Networks and Embedded Microcontroller Architecture," *Atmel, ARM*.
- [19]. K. Jiang, P. Eles, Z. Peng, "Optimization of Message Encryption for Distributed Embedded Systems with Real-Time Constraints," *IEEE 14th International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS)*, pp. 243-248, 2011.
- [20]. J. Peddersen, Sri Parameswaran, "Energy Driven Application Self-Adaptation at Run-time," 20th International Conference on VLSI Design. Held jointly with 6th International Conference on Embedded Systems, pp. 385-390, 2007.
- [21]. T. Xie, X. Qin, "Scheduling Security-Critical Real-Time Applications on Clusters," *IEEE Transaction on Computers*, vol. 55, pp. 864-879, 2006.
- [22]. Song S. Kai Hwang, Yu-Kwong Kwok, "Risk-Resilient Heuristics and Genetic Algorithms for Security-Assured Grid Job Scheduling," *IEEE Transactions on Computers*, vol. 55, issue. 6, pp. 703-719, 2006.